W2D4

Created time	October 15, 2026 4:09 PM

Q1. For each requirement, choose data lake, data warehouse, or lakehouse.

Requirement	Choice	Reason
Large raw mixed data	Data lake	As it stores raw data
Controlled SQL-ready business data	Data warehouse	Structured, analytical use
Flexible storage plus managed analytical tables	Lakehouse	Its a comination of lake-style storage with warehouse-style table capabilities
Raw website events	Data lake	Raw events can be stored

Q2.

MySQL order extract —-Data Warehouse
supplier CSV files — Data Warehouse(same format) else Lakehouse
website JSON events —— Data Lake /Lakehouse
monthly finance table —— Data Warehouse/Lakehouse
daily sales summary ——- Data Warehouse/Lakehouse

Q3.

Area	Data Lake	Data Warehouse	Lakehouse
Raw data support	Strong	not preferred	Strong
Strong structured reporting	not preferred	Strong	Strong
Flexible data shapes	Strong	Not preferred	Strong
Table-style analytics	Usually not preferred	Strong	Strong

Q4. D. Run controlled business reporting over trusted structured data

Q5.

A. Data is stored first and interpreted when it is read.—- Schema-on-read
B. Data must match an expected structure before it is accepted into the target. —-Schema-on-write
C. A raw JSON area keeps different optional fields. —- Schema-on-read
D. A controlled reporting table requires known columns and types.—-Schema-on-write

Q6.

Bronze: Bronze prioritises preserving what arrived. Keep the received records close to the source form so the team can inspect or replay them later.

Silver: Improve data quality. Validate, clean, standardise, and flag invalid records

Q7

```
import pandas as pd

batch_1 = pd.DataFrame(
[
(101, "London", 45.50, "complete"),
(102, "Manchester", -5.00, "complete"),
],
columns=[
"order_id",
"city",
"amount_gbp",
"status",
],
)

batch_2 = pd.DataFrame(
[
(103, " Bristol ", 30.00, "complete"),
(104, "Leeds", 20.00, "cancelled"),
],
columns=[
"order_id",
"city",
"amount_gbp",
"status",
],
)

batch_3 = pd.DataFrame(
[
(101, "London", 45.50, "complete"),
(105, "Manchester", 51.00, "complete"),
],
columns=[
"order_id",
"city",
"amount_gbp",
"status",
],
)
```

```
bronze = pd.concat(
[batch_1, batch_2, batch_3],
ignore_index=True,
)

silver = bronze[
bronze["amount_gbp"] > 0
].copy()

silver["city"] = (
silver["city"].str.strip()
)

silver = silver.drop_duplicates(
subset=["order_id"],
keep="first",
)

print("Bronze rows:", len(bronze))
print("Silver rows:", len(silver))
print(silver)
```

Q8. Which record belongs most naturally in Silver?

C. A cleaned and deduplicated order row

Q9.

```
completed = silver[
silver["status"] == "complete"
]

gold = (
completed
.groupby(
"city",
as_index=False,
)
.agg(
revenue_gbp=(
"amount_gbp",
"sum",
)
)
.sort_values(
"revenue_gbp",
ascending=False,
```

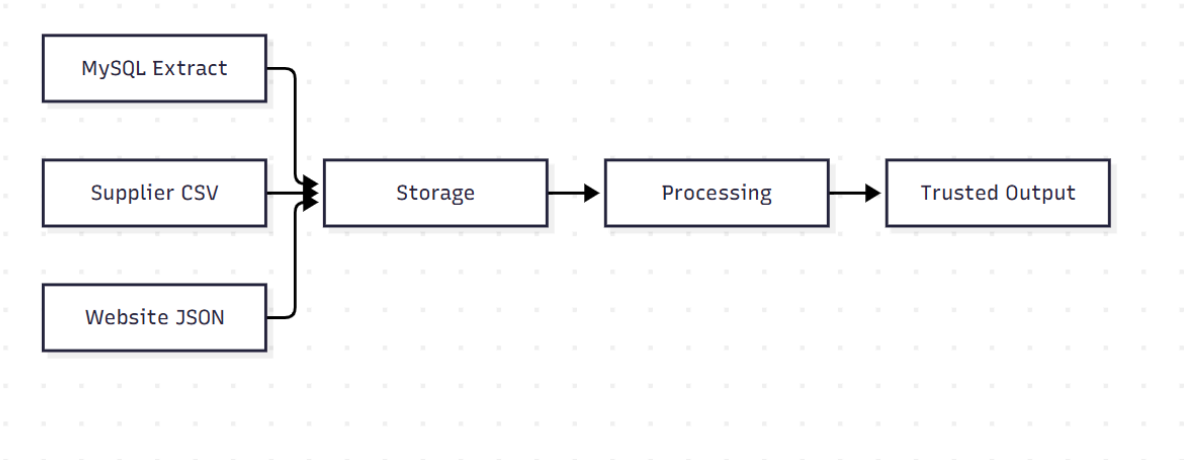
)
)

print(gold)

Q10.

MySQL orders table → Structured
JSON website event → Semi-structured
CSV supplier file → Structured
Product photo → Unstructured
PDF supplier contract → Unstructured

Q11



Q12.

Which output is the strongest Gold example?

A. Daily revenue by city for reporting

Q13.

A. Files kept for later use —- Storage
B. CPU and memory used to run a transformation —-Compute
C. Stored analytical table data —-Storage
D. Processing resources used for a query —-Compute

Q14.

Separating compute and storage is useful because BritMart can scale processing power independently of the data it stores.

- Cost savings: During the day, BritMart can use minimal compute resources while data is being collected. At night, it can increase compute resources for the large transformation and reduce them afterward.

- Faster processing: More compute resources can be allocated to handle the nightly workload efficiently.
- No data movement required: The stored data remains in place when compute resources change, avoiding unnecessary copying or migration.
- Independent scaling: Storage can grow as new data arrives without requiring more processing power, and compute can grow without increasing storage capacity.
- Better resource utilisation: BritMart pays for processing capacity when it is needed rather than keeping powerful resources running all day.

Example: BritMart stores orders throughout the day, then increases compute capacity overnight to clean and transform those orders. Once the job finishes, it scales compute back down while keeping the data stored.

And separating compute and storage improves flexibility, scalability, and cost efficiency because each resource can be managed independently according to demand.

Q15

Dataset	Purpose	Main columns	Source	Owner/team	Last update
bronze_orders	Received order data	order_id, city, amount, status	Source batches	Data Engineering	2026-08-01
silver_orders	Clean reusable orders	order_id, city, amount, status	bronze_orders	Data Engineering	2026-08-01
gold_daily_sales	Reporting summary	city, revenue_gbp	silver_orders	Analytics/Data	2026-08-01

Q16. B. It describes data so people and systems can understand what the data represents.

Q17.

Bronze — Raw data

Store the record as received, without changing its values.

{
"order_id": 9001,
"city": " London ",
"amount_gbp": 50,
"status": "complete"
}

Purpose: Preserve the original data for auditing, recovery, and reprocessing.

Silver — Cleaned and validated data

Trim spaces from the city and standardise the status value according to the agreed business rules.

{
"order_id": 9001,
"city": "London",
"amount_gbp": 50.00,
"status": "Completed"
}

Purpose: Ensure the record has valid values, correct data types, and consistent formatting.

3. Gold — Business-ready data

Use the validated record in a reporting table, such as total completed sales by city.

city	completed_order_count	total_sales_gbp
London	1	50.00

Q18.

1. Inconsistent and unreliable data: Gold is built directly from raw Bronze data without a shared cleaning and validation step. Missing values, duplicates, and inconsistent formats may produce inaccurate reports.
2. Duplicated work and maintenance: Repeating different cleaning rules across five reports creates inconsistent results. If a business rule changes, the team must update multiple reports, increasing maintenance effort and the risk of errors.

Better design: Clean and validate data once in the Silver layer, then build the Gold layer from Silver data. This creates consistent, trusted data for all reports.

Q19 Reconcile the Layers

Step 1: Calculate the difference between Bronze and Silver.

- Bronze rows = 1,000
- Silver rows = 960
- Rows removed = \(1000 - 960 = 40\)

Step 2: Add the team's explanations.

- Invalid amounts = 25 rows
- Duplicates = 15 rows
- Total explained = \(25 + 15 = 40\) rows

Step 3: Check reconciliation.

Item	Row count
Bronze rows	1,000

Item	Row count
Invalid amount rows	25
Duplicate rows	15
Total rows removed	40
Expected Silver rows	960
Actual Silver rows	960
Unexplained difference	0

Conclusion: The numbers reconcile because \(25 + 15 = 40\), matching the difference between Bronze and Silver.

The team should still verify that the 25 invalid rows and 15 duplicate rows are distinct, correctly counted, and recorded in its data-quality logs. If those categories overlap, the reconciliation would need further investigation.

Q20.

